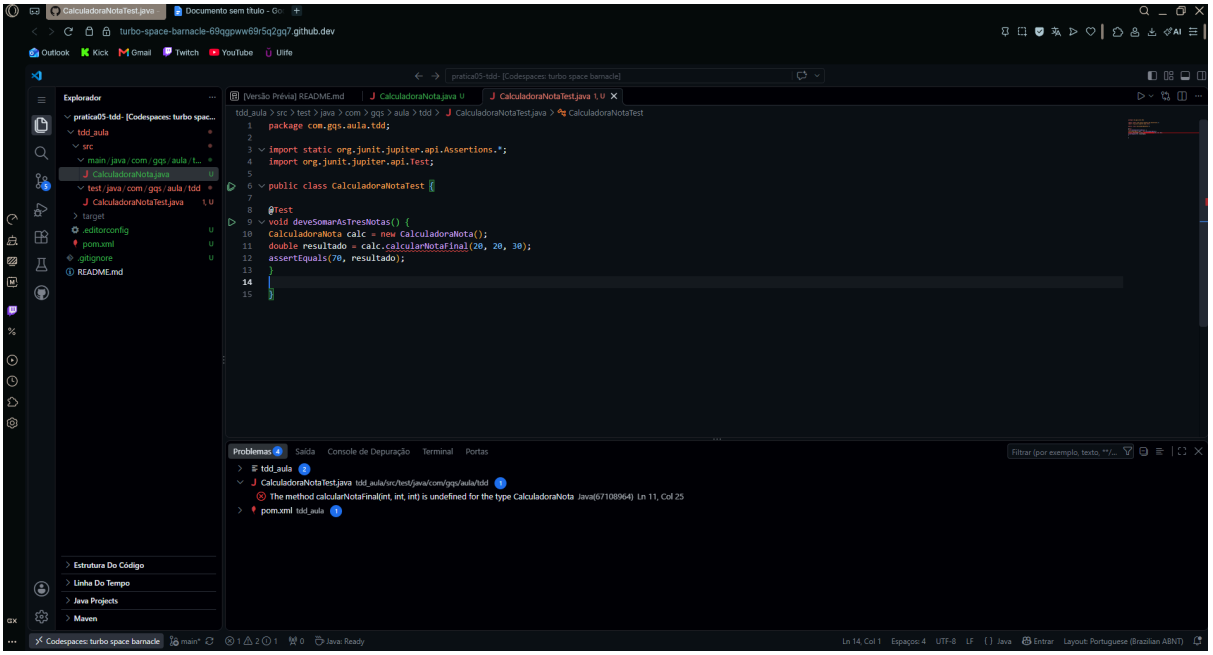
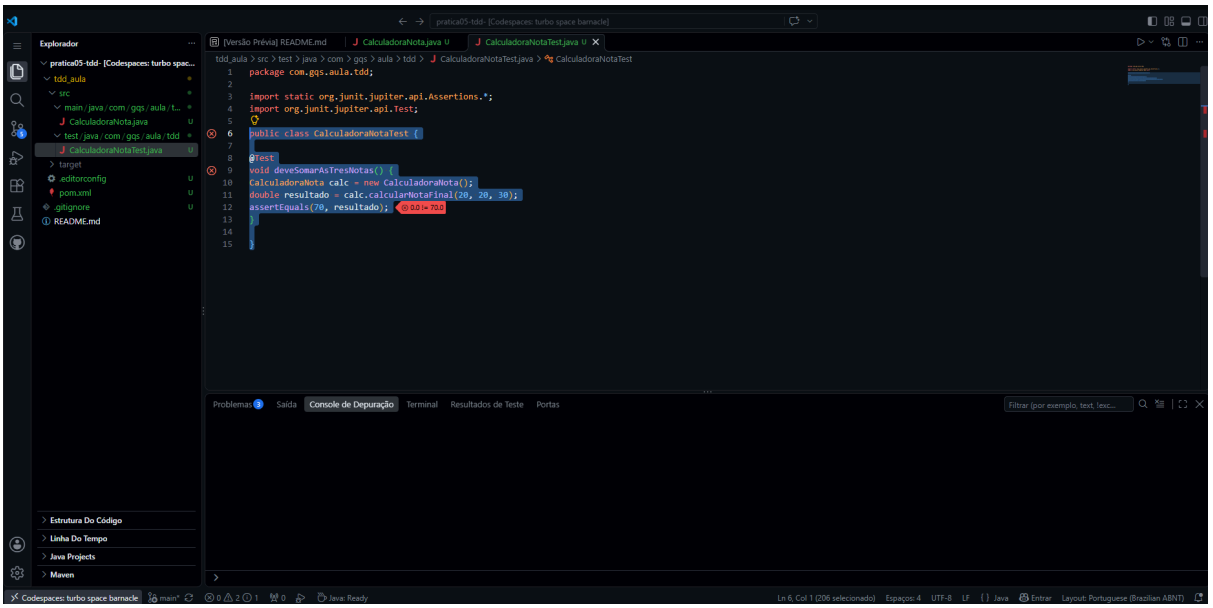
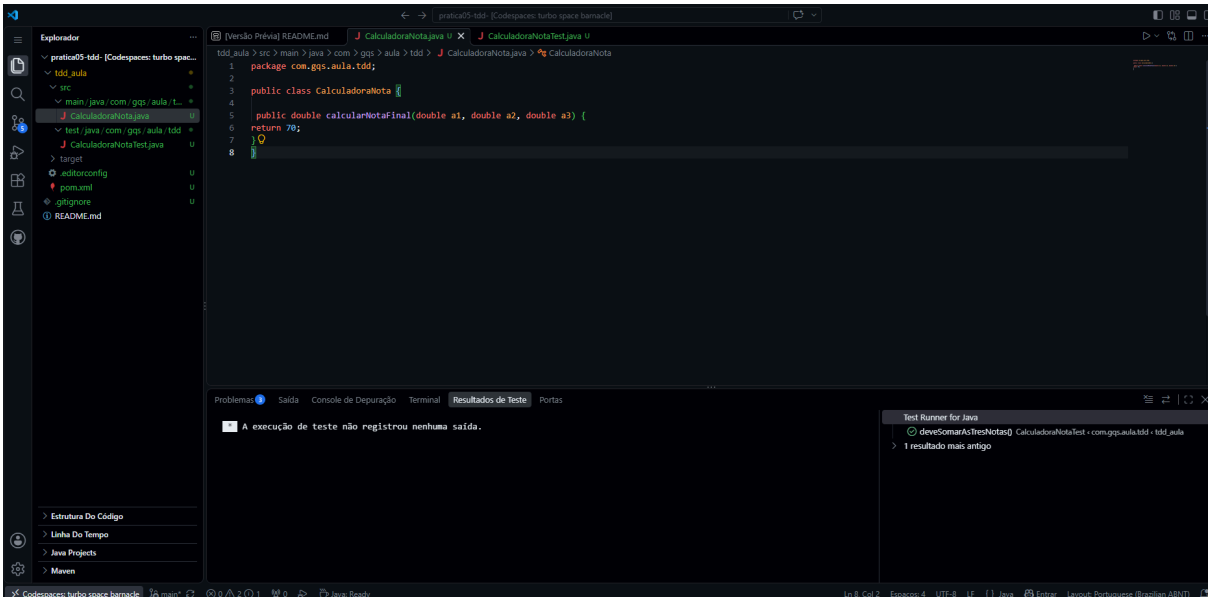
calcularNotaFinal fica sublinhado em vermelho: o método ainda não existe e o código não compila.



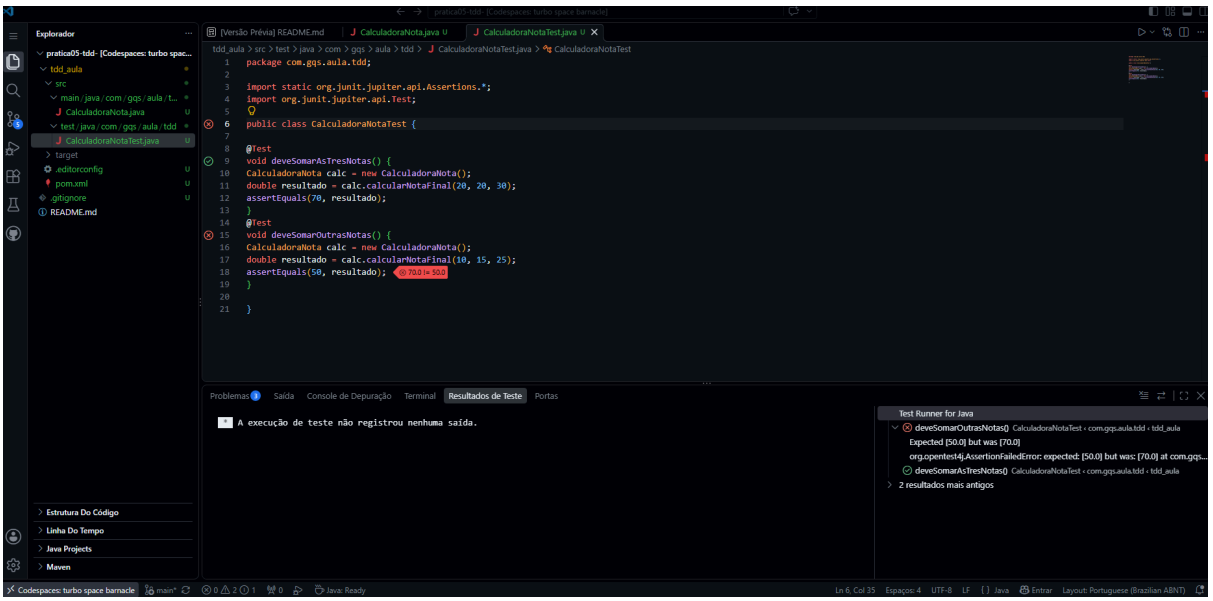
expected: <70.0> but was: <0.0>



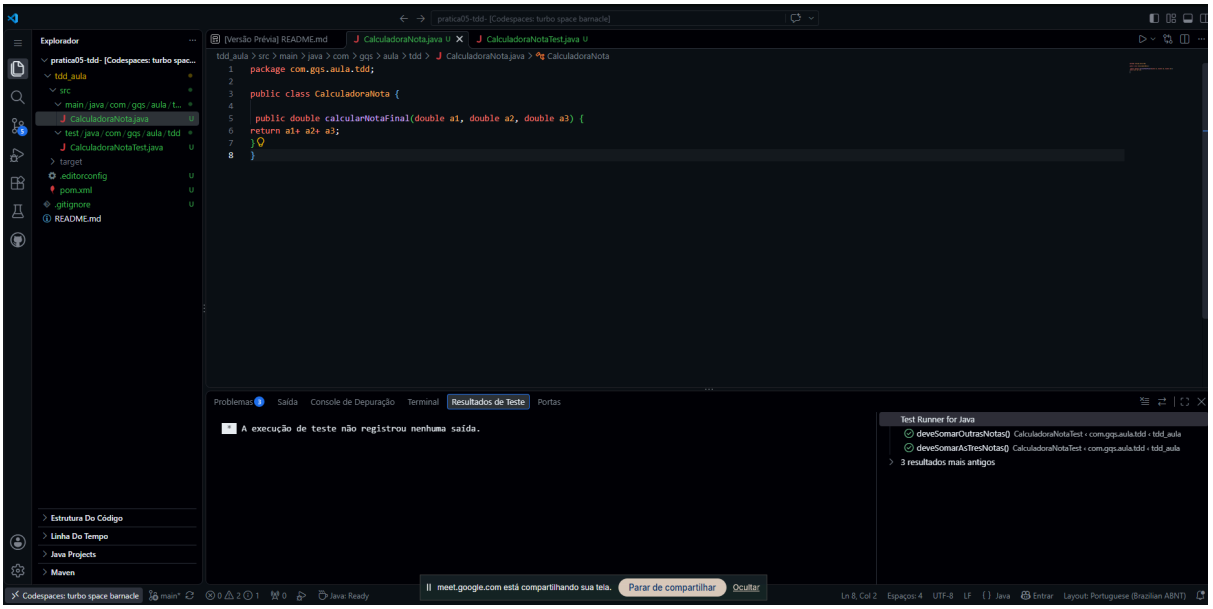
VERDE O mínimo para passar: retornar 70



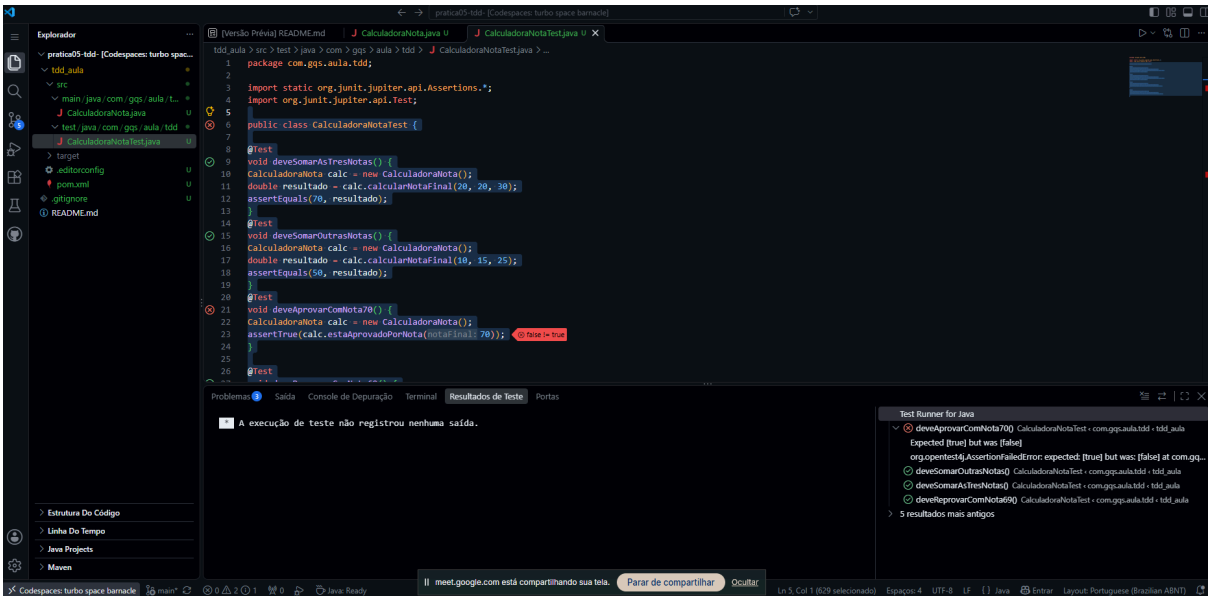
expected: <50.0> but was: <70.0>



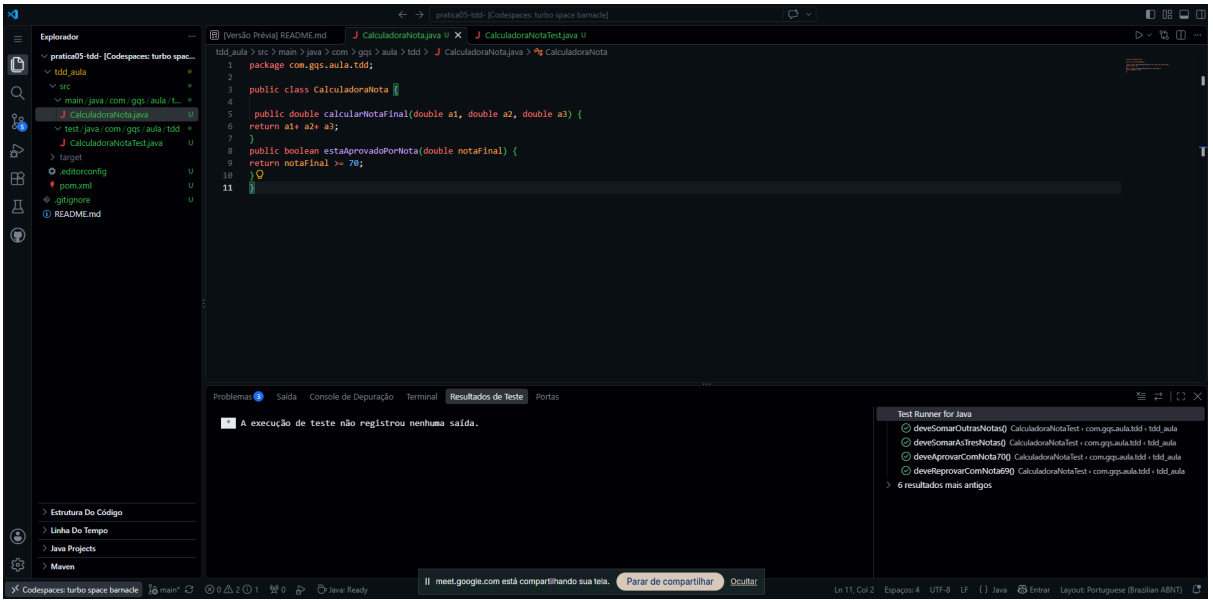
Agora sim, a soma de verdade. Rode todos: os dois passam.

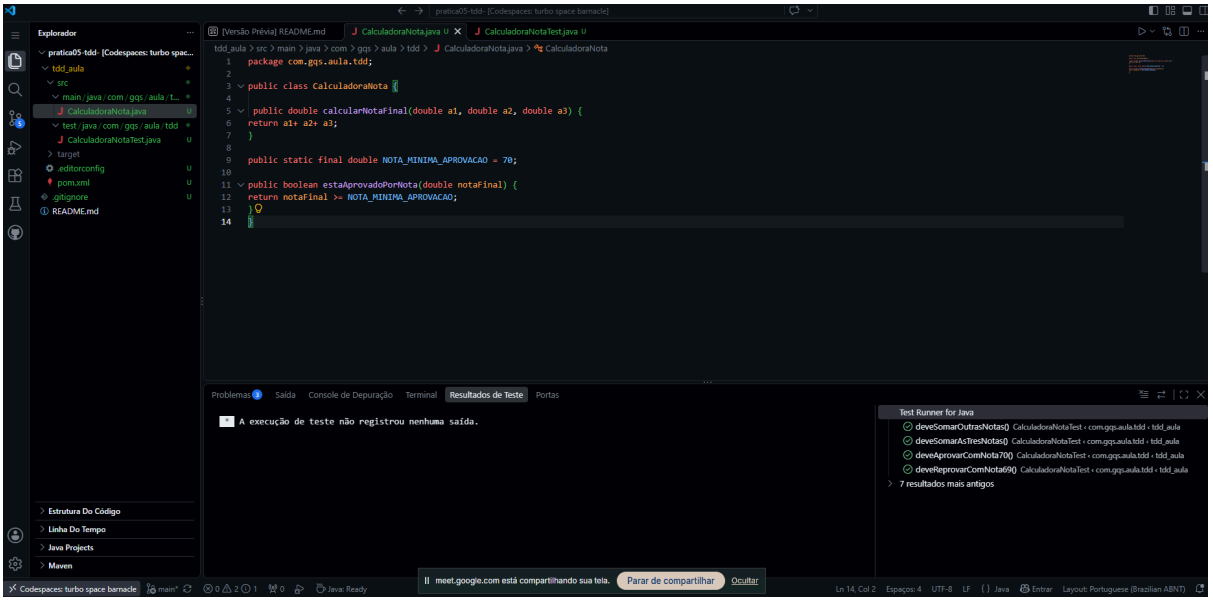


VERMELHO Teste na fronteira: 70 e 69

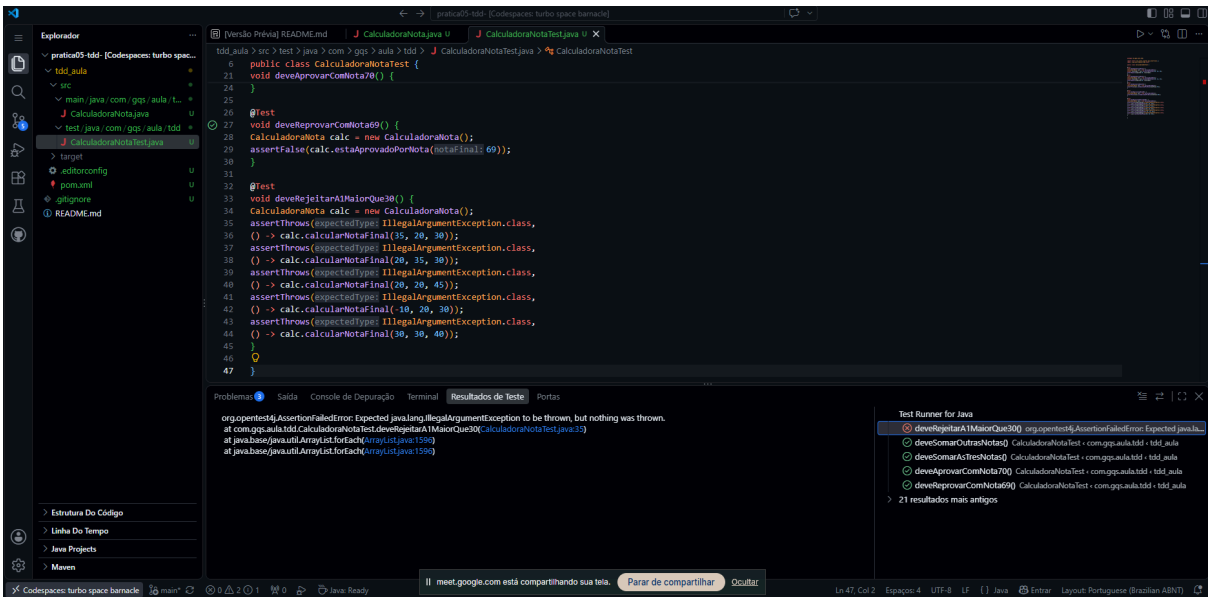


Faça passar e dê nome ao número 70

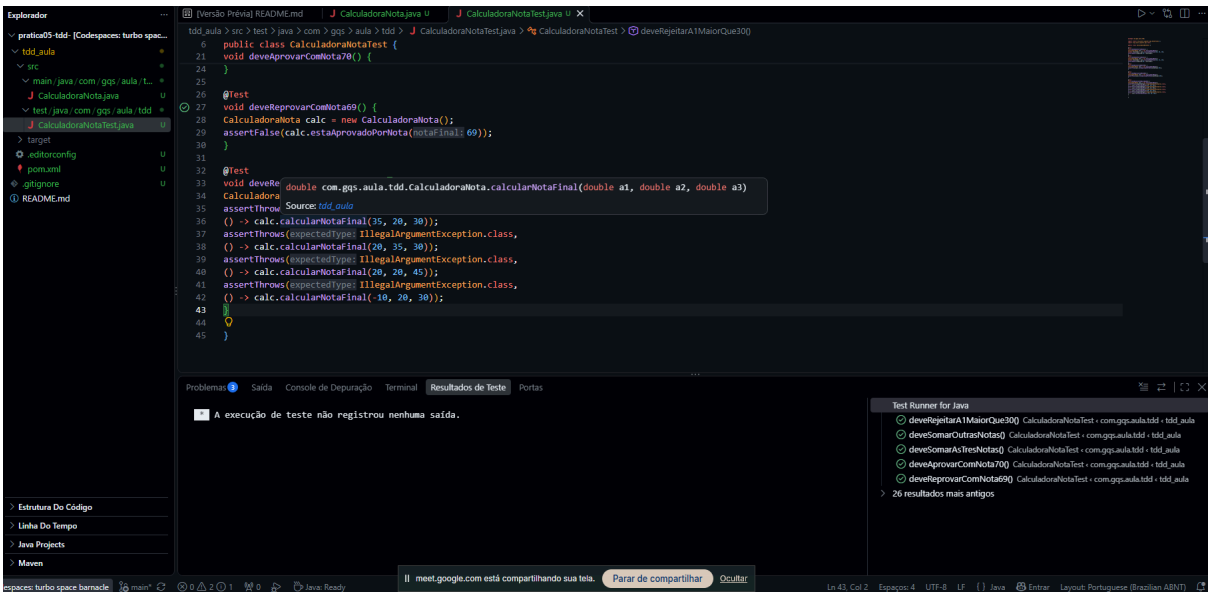




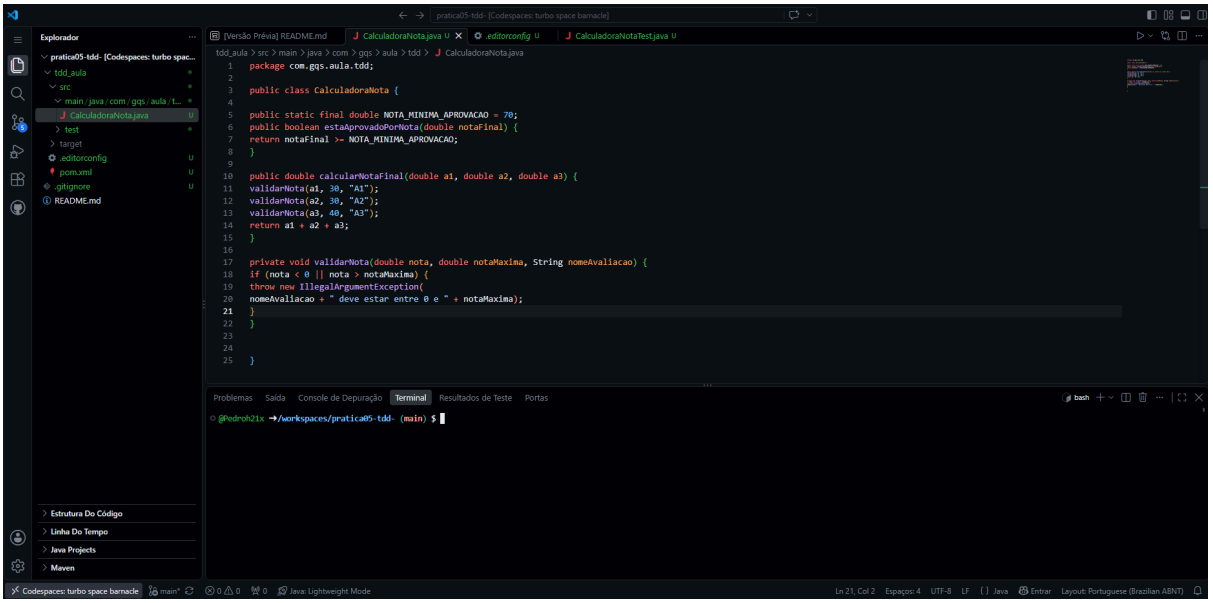
Rode: os quatro de rejeição, incluindo o modelo, falham com “nothing was thrown”. O das máximas passa.



Rode Todos os testes, até ficarem verdes.



Três if parecidos viram um método



```
package com.gqs.aula.tdd;

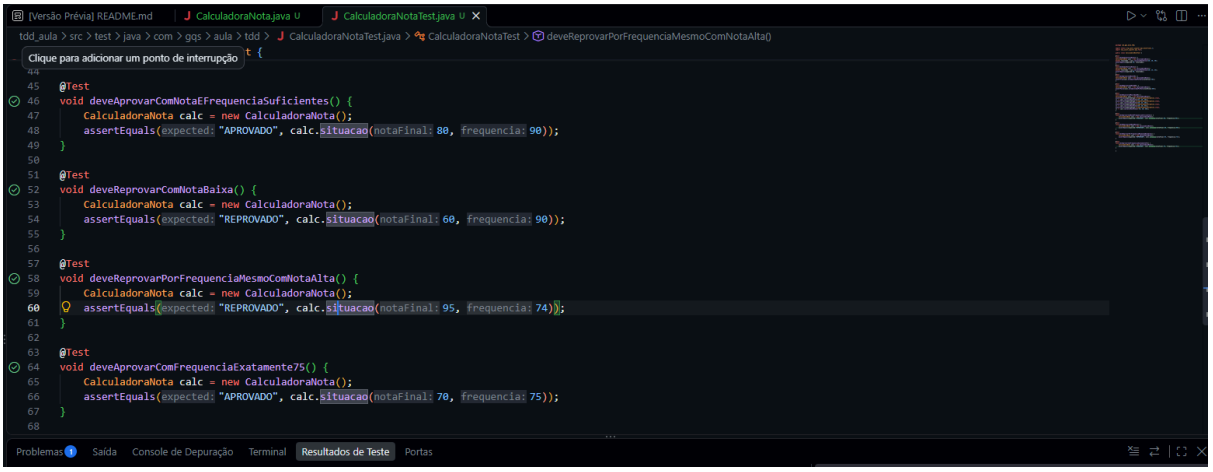
public class CalculadoraNota {

public static final double NOTA_MINIMA_APROVACAO = 70;
public boolean estaAprovadoPorNota(double notaFinal) {
return notaFinal >= NOTA_MINIMA_APROVACAO;
}

public double calcularNotaFinal(double a1, double a2, double a3) {
validarNota(a1, 30, "A1");
validarNota(a2, 30, "A2");
validarNota(a3, 40, "A3");
return a1 + a2 + a3;
}

private void validarNota(double nota, double notaMaxima, String nomeAvaliacao) {
if (nota < 0 || nota > notaMaxima) {
throw new IllegalArgumentException(
nomeAvaliacao + " deve estar entre 0 e " + notaMaxima);
}
}


}
```



```
Clique para adicionar um ponto de interrupção
t {

@Test
void deveAprovarComNotaEFrequenciaSuficientes() {
    CalculadoraNota calc = new CalculadoraNota();
    assertEquals(expected: "APROVADO", calc.situacao(notaFinal: 80, frequencia: 90));
}

@Test
void deveReprovarComNotaBaixa() {
    CalculadoraNota calc = new CalculadoraNota();
    assertEquals(expected: "REPROVADO", calc.situacao(notaFinal: 60, frequencia: 90));
}

@Test
void deveReprovarPorFrequenciaMesmoComNotaAlta() {
    CalculadoraNota calc = new CalculadoraNota();
    assertEquals(expected: "REPROVADO", calc.situacao(notaFinal: 95, frequencia: 74));
}

@Test
void deveAprovarComFrequenciaExatamente75() {
    CalculadoraNota calc = new CalculadoraNota();
    assertEquals(expected: "APROVADO", calc.situacao(notaFinal: 70, frequencia: 75));
}
```



```
public class CalculadoraNota {
public boolean estaAprovadoPorNota(double notaFinal) {
return notaFinal >= NOTA_MINIMA_APROVACAO;
}

public double calcularNotaFinal(double a1, double a2, double a3) {
validarNota(a1, notaMaxima: 30, nomeAvaliacao: "A1");
validarNota(a2, notaMaxima: 30, nomeAvaliacao: "A2");
validarNota(a3, notaMaxima: 40, nomeAvaliacao: "A3");
return a1 + a2 + a3;
}

private void validarNota(double nota, double notaMaxima, String nomeAvaliacao) {
if (nota < 0 || nota > notaMaxima) {
throw new IllegalArgumentException(
nomeAvaliacao + " deve estar entre 0 e " + notaMaxima);
}
}

public String situacao(double notaFinal, double frequencia) {
    if (estaAprovadoPorNota(notaFinal) && frequencia >= 75) {
        return "APROVADO";
    }
    return "REPROVADO";
}
}
```

Altere situacao para retornar os três textos.

